FACE Prep - MongoDB Certification Project

Student Name: Ananta Krishnan B
Registration Number: 23BCE1397
Campus: VIT Chennai Cmapus
Project Date: June 18, 2026

---

1. Project Title

Movied: A High-Performance Movie Database Management System (API & Swagger UI Documentation)

---

2. Project Outcome

The objective of this project was to establish a fully functional Movie Database Management System utilizing **MongoDB** for document storage and relationships, and **FastAPI** for an asynchronous backend service.

Instead of a CLI interface, the system leverages FastAPI's built-in **Swagger UI (OpenAPI Specification)** to serve as the interactive testing documentation. Users and evaluators can execute CRUD operations and retrieve reports directly from their web browsers.

Key Deliverables Met:

1. **Database Integration**: Set up asynchronous connectivity to MongoDB using `motor`.
2. **Interactive Documentation (Swagger UI)**: Configured OpenAPI routes serving Swagger UI at `/docs` for browser-based testing.
3. **Standard CRUD Layer**: Created full Create, Read, Update, and Delete endpoints for 5 distinct database entities:
   - **Movies**: Storing details like title, release year, duration, language, rating, and box office.
   - **Actors**: Storing actor name, birth year, and nationality.
   - **Directors**: Storing director name, birth year, and nationality.
   - **Genres**: Categorizing movies into distinct genres (Sci-Fi, Drama, Biography, etc.).
   - **Reviews**: Recording viewer names, rating scores, and textual reviews.
4. **Relational Reference Design**: Managed relationships across document collections:
   - *One Director → Many Movies* (`director_id` reference in Movie)

- *One Genre* → *Many Movies* (`genre_id` reference in Movie)
- *Many Actors* ↔ *Many Movies* (`actor_ids` list reference in Movie)
- *One Movie* → *Many Reviews* (`movie_id` reference in Review)

5. **Trigger Recalculations**: Automatically updated a movie's average rating whenever reviews are added, modified, or deleted.
6. **Advanced Aggregations**: Implemented 5 complex aggregation reports fetching multi-layer relational data from MongoDB.

---

3. Tools and Versions Used

The system is built on a modern, asynchronous Python backend stack and a local MongoDB instance. The versions verified in the environment are:

Tool / Dependency	Version	Description
MongoDB Database Server	`v8.2.4`	Document-oriented NoSQL database engine
Python Interpreter	`v3.9.24`	Language base for backend logic
FastAPI	`v0.128.8`	Modern, fast, ASGI web framework
Motor	`v3.7.1`	Non-blocking MongoDB driver for asyncio python
Pydantic	`v2.13.4`	Data validation and parsing using Python type hints
Uvicorn	`v0.39.0`	High-performance ASGI server implementation

---

4. How to Run the Project & Access Swagger UI

To start the services and launch the interactive testing suite, execute the following commands in the terminal:

Step 1: Start MongoDB

Start the MongoDB daemon using a local database storage path:

```
mongod --dbpath ./mongodb_data --port 27017 --fork --logpath
./mongodb_data/mongod.log
```

Step 2: Seed the Database

Prepopulate the MongoDB collections with sample actors, directors, genres, movies, and user reviews:

```
./venv/bin/python seed_db.py
```

Step 3: Run the FastAPI Application

Start the Uvicorn web server hosting the FastAPI backend:

```
./venv/bin/uvicorn app.main:app --host 127.0.0.1 --port 8000 --reload
```

Step 4: Open Swagger UI

Open your web browser and navigate to:

```
http://127.0.0.1:8000/docs
```

You will see the full API endpoint catalog sorted by entity tags (Directors, Genres, Actors, Movies, Reviews, Reports) ready for interactive execution.

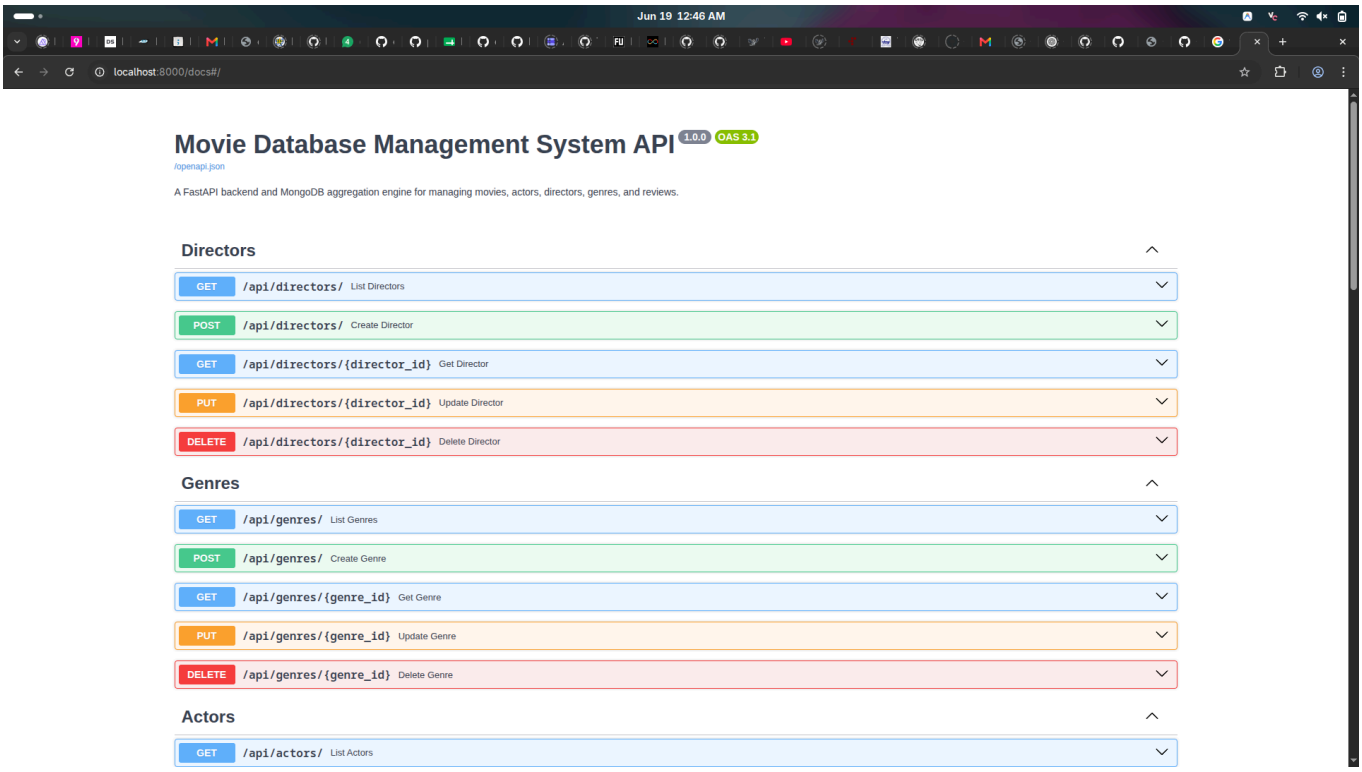
---

5. Swagger UI Execution & API Outcomes

This section documents the endpoints and responses retrieved when executing operations through the Swagger UI dashboard.

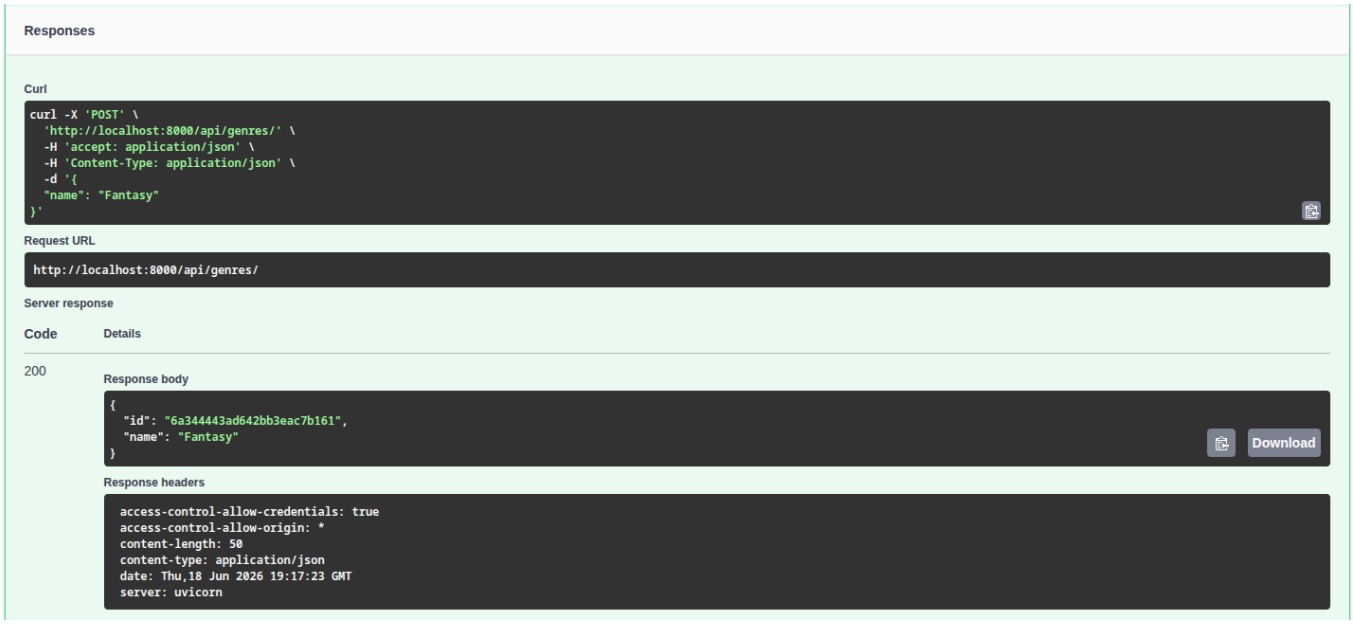
5.1 Swagger UI Overview Documentation

Displays all endpoints defined in the application, grouped by tags.



5.2 Genre Operations via Swagger UI

Shows adding a new Genre document.



Response JSON:

```
{ "id": "6a344443ad642bb3eac7b161", "name": "Fantasy" }
```

5.3 Director & Actor Operations via Swagger UI

Shows adding a Director and Actor document.

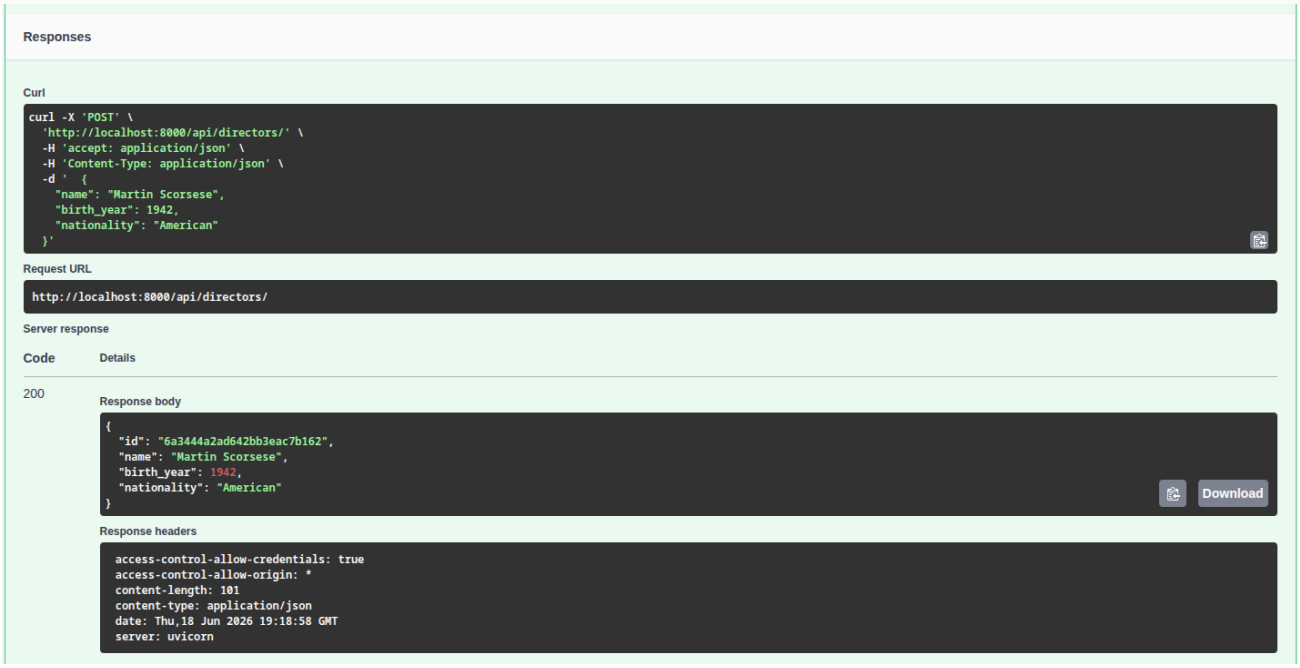
1. POST /api/directors/ (Martin Scorsese)

- **Request Body**:

```
{
  "name": "Martin Scorsese",
  "birth_year": 1942,
  "nationality": "American"
}
```

- **Response JSON**:

```
{ "id": "6a3444a2ad642bb3eac7b162", "name": "Martin Scorsese",
"birth_year": 1942, "nationality": "American" }
```



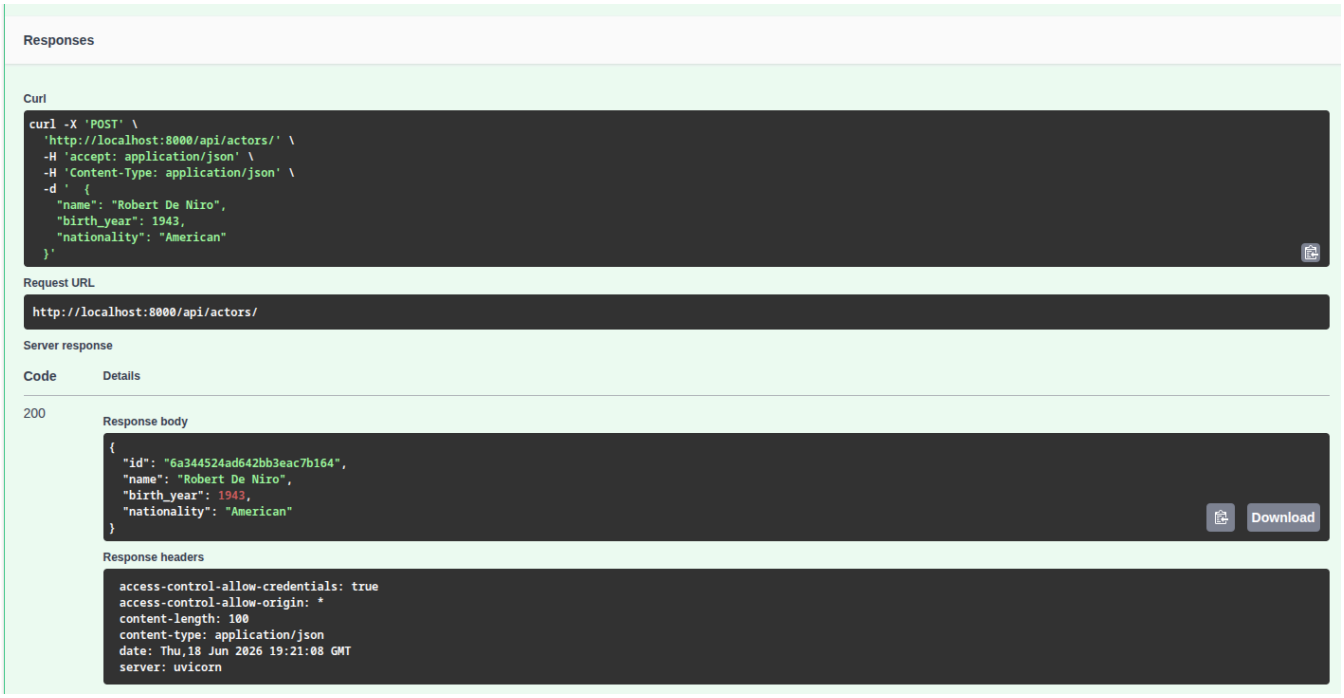
2. POST /api/actors/ (Robert De Niro)

- **Request Body**:

```
{
  "name": "Robert De Niro",
  "birth_year": 1943,
  "nationality": "American"
}
```

- **Response JSON**:

{ "id": "6a344524ad642bb3eac7b164", "name": "Robert De Niro",
"birth_year": 1943, "nationality": "American" }



5.4 Movie CRUD Operations & Relational Resolution via Swagger UI

Demonstrates Movie creation and reading. Crucially, executing `GET /api/movies/{movie_id}/detailed` shows **Relational Lookup Resolution** where Director, Genre, and Actors references are resolved using MongoDB aggregations in a single query.

1. POST /api/movies/ (Create Memento)

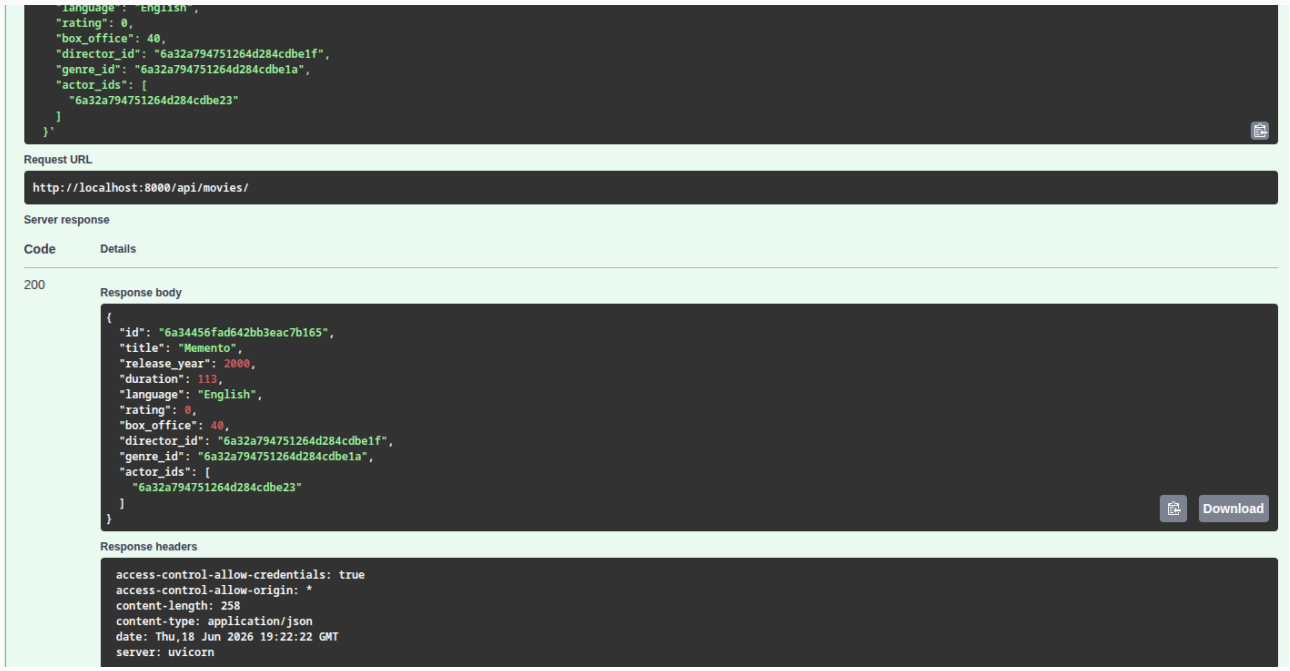
- **Request Body**:

{
"title": "Memento",
"release_year": 2000,
"duration": 113,
"language": "English",
"rating": 0,
"box_office": 40,
"director_id": "6a32a794751264d284cdbe1f",
"genre_id": "6a32a794751264d284cdbe1a",
"actor_ids": [
"6a32a794751264d284cdbe23"

```
  ]
}
```

- **Response JSON**:

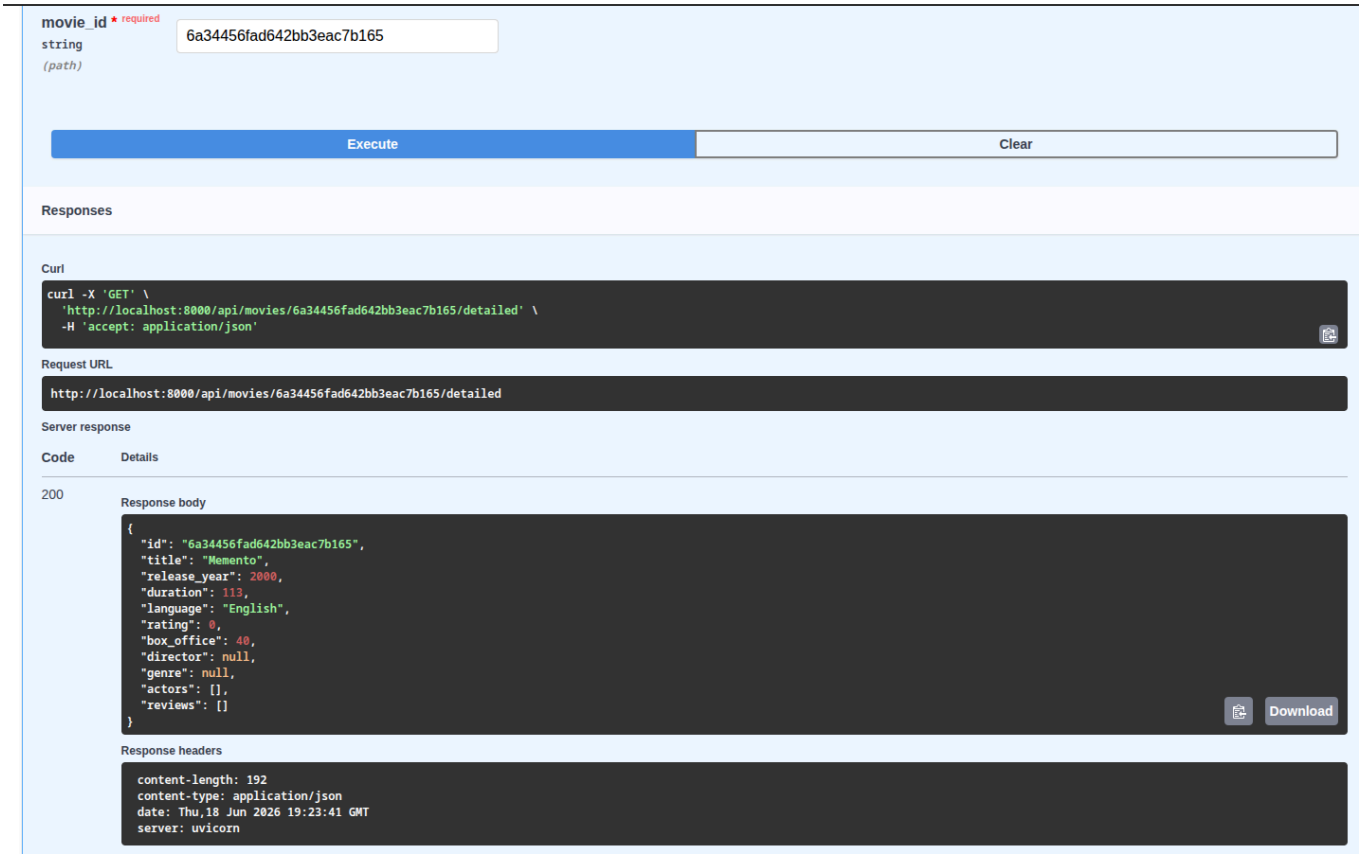
```
{ "id": "6a34456fad642bb3eac7b165", "title": "Memento", "release_year": 2000, "duration": 113, "language": "English", "rating": 0, "box_office": 40, "director_id": "6a32a794751264d284cdbe1f", "genre_id": "6a32a794751264d284cdbe1a", "actor_ids": [ "6a32a794751264d284cdbe23" ] }
```



2. GET /api/movies/{movie_id}/detailed (Detailed lookup for Memento)

- **Response JSON**:

```
{ "id": "6a34456fad642bb3eac7b165", "title": "Memento", "release_year": 2000, "duration": 113, "language": "English", "rating": 0, "box_office": 40, "director": null, "genre": null, "actors": [], "reviews": [] }
```



- **Explanation**: Creating a movie stores string ObjectId references (`director_id` , `genre_id` , `actor_ids`). Querying the detailed endpoint executes a MongoDB `$lookup` pipeline on the backend, joining documents across collections to return fully resolved relationships.

5.5 Review Submission & Dynamic Rating Recalculation

Demonstrates writing reviews and verifying rating updates.

1. POST /api/reviews/ (Add Review 1)

- **Request Body**:

```
{
  "movie_id": "6a34456fad642bb3eac7b165",
  "reviewer_name": "John Doe",
  "rating": 8.5,
  "review_text": "Incredible backwards-structured plot! Keeps you
guessing."
}
```

2. POST /api/reviews/ (Add Review 2)

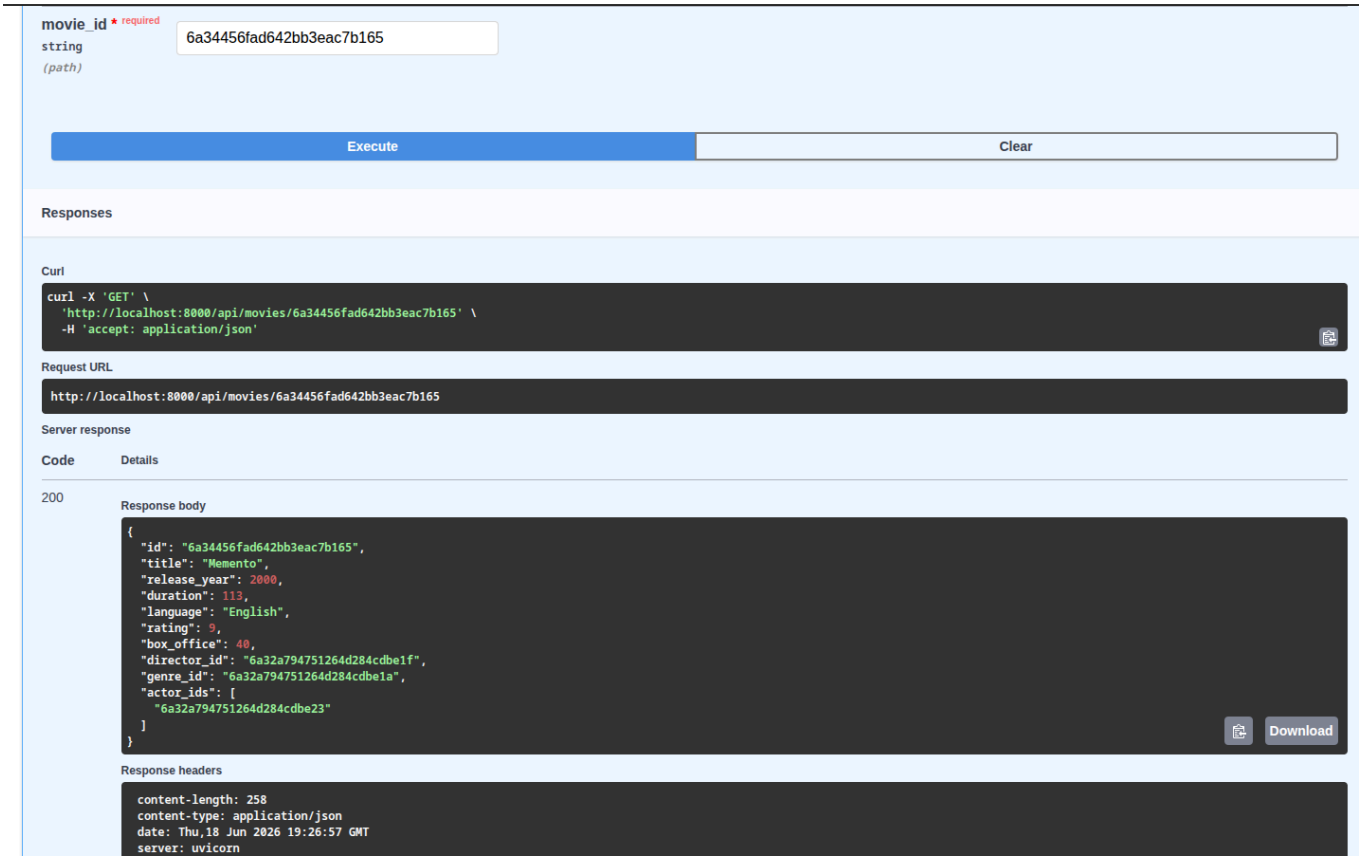
- **Request Body**:

```
{
  "movie_id": "6a34456fad642bb3eac7b165",
  "reviewer_name": "Jane Miller",
  "rating": 9.5,
  "review_text": "One of Nolan's finest works. Unforgettable."
}
```

3. GET /api/movies/{movie_id} (Check Updated Rating)

- **Response JSON**:


```
{ "id": "6a34456fad642bb3eac7b165", "title": "Memento", "release_year": 2000, "duration": 113, "language": "English", "rating": 9, "box_office": 40, "director_id": "6a32a794751264d284cdbe1f", "genre_id": "6a32a794751264d284cdbe1a", "actor_ids": [ "6a32a794751264d284cdbe23" ] }
```

- **Explanation**: Adding reviews triggers an aggregation process that calculates the average review score (`(8.5 + 9.5) / 2 = 9.0`) and updates the movie document in real-time.

5.6 Advanced MongoDB Aggregation Pipelines (Reports)

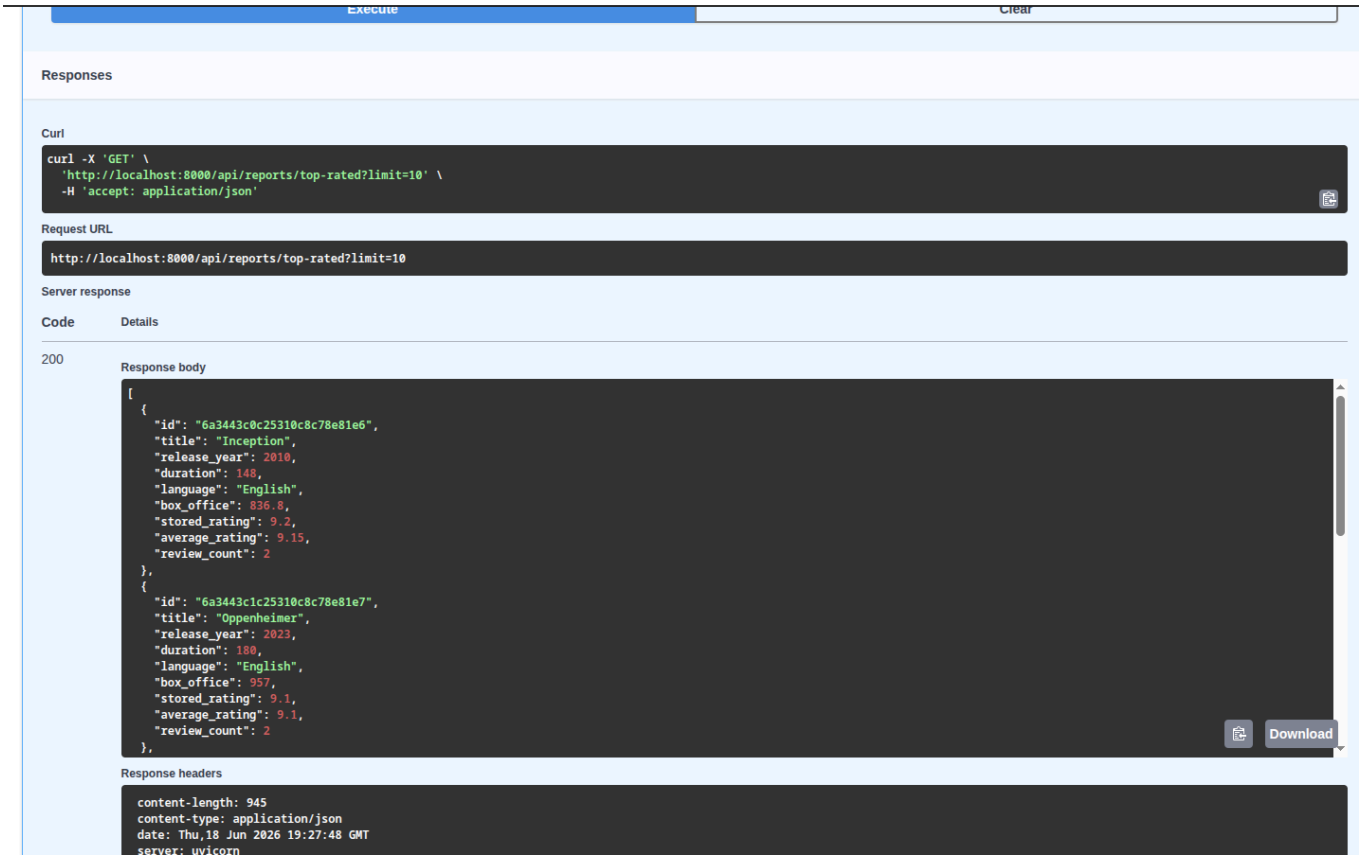
Executing reporting pipelines under `/api/reports/` tag:

1. GET /api/reports/top-rated (Top-Rated Movies)

Sorts movies dynamically based on average review scores.

- **Response JSON**:

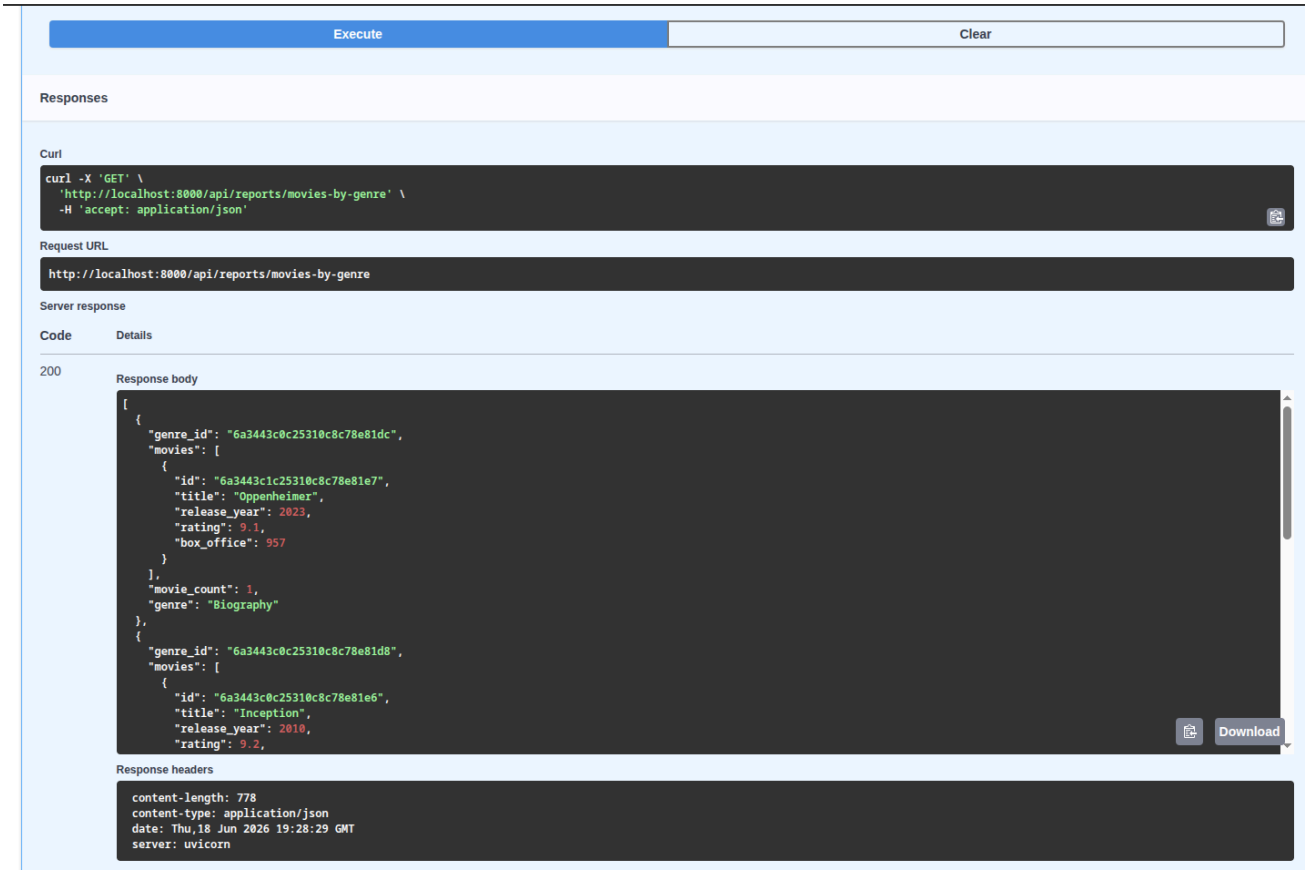

```
[ { "id": "6a3443c0c25310c8c78e81e6", "title": "Inception",
"release_year": 2010, "duration": 148, "language": "English",
"box_office": 836.8, "stored_rating": 9.2, "average_rating": 9.15,
"review_count": 2 }, { "id": "6a3443c1c25310c8c78e81e7", "title":
"Oppenheimer", "release_year": 2023, "duration": 180, "language":
"English", "box_office": 957, "stored_rating": 9.1, "average_rating":
9.1, "review_count": 2 }, { "id": "6a3443c1c25310c8c78e81e8", "title":
"Dune: Part Two", "release_year": 2024, "duration": 166, "language":
"English", "box_office": 712.4, "stored_rating": 9.1, "average_rating":
9.05, "review_count": 2 }, { "id": "6a34456fad642bb3eac7b165", "title":
"Memento", "release_year": 2000, "duration": 113, "language": "English",
"box_office": 40, "stored_rating": 9, "average_rating": 9,
"review_count": 2 }, { "id": "6a3443c1c25310c8c78e81e9", "title":
"Interstellar", "release_year": 2014, "duration": 169, "language":
"English", "box_office": 731, "stored_rating": 8.9, "average_rating":
8.9, "review_count": 1 } ]
```



2. GET /api/reports/movies-by-genre (Movies Grouped by Genre)

- **Response JSON**:

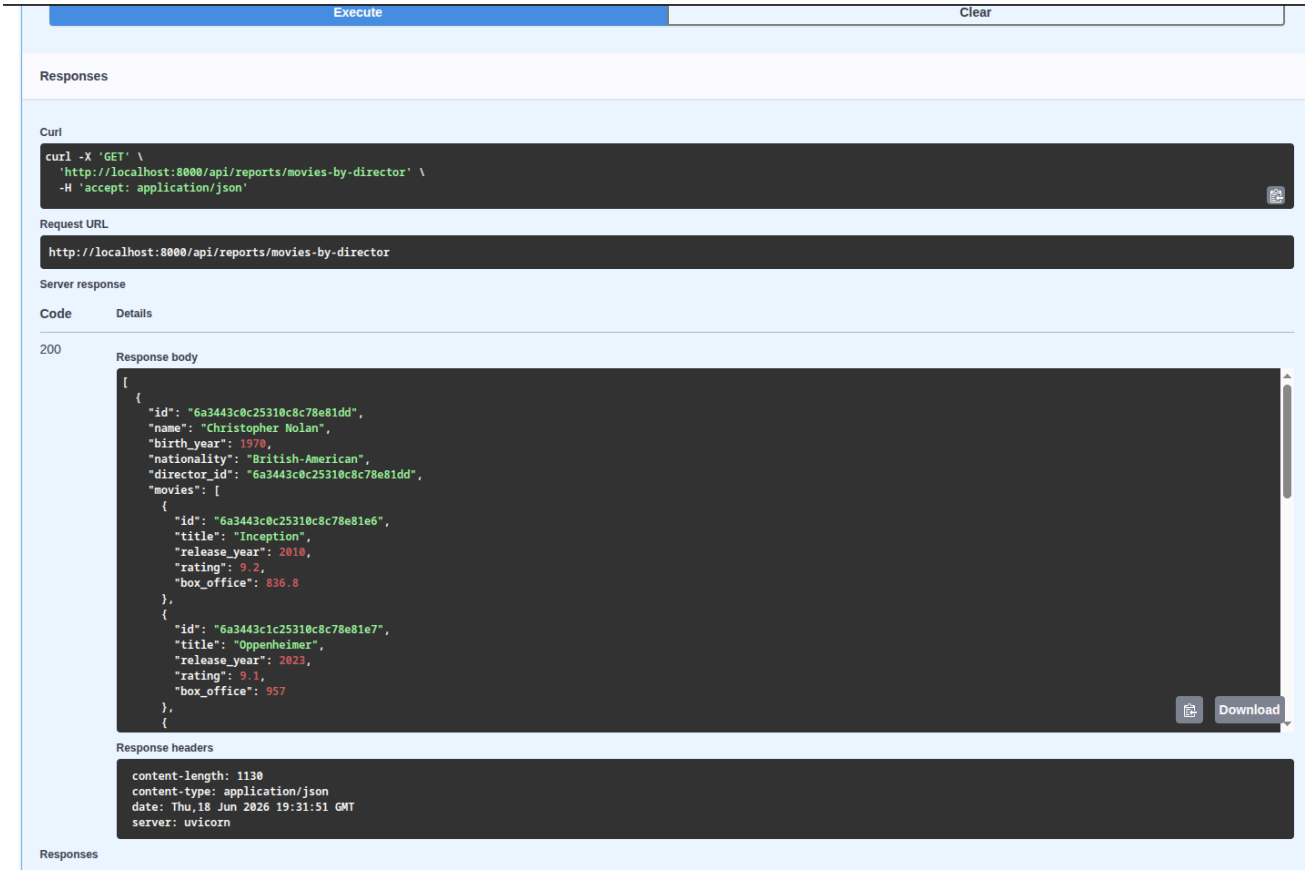

```
[ { "genre_id": "6a3443c0c25310c8c78e81dc", "movies": [ { "id":
"6a3443c1c25310c8c78e81e7", "title": "Oppenheimer", "release_year": 2023,
"rating": 9.1, "box_office": 957 } ], "movie_count": 1, "genre":
"Biography" }, { "genre_id": "6a3443c0c25310c8c78e81d8", "movies": [ {
"id": "6a3443c0c25310c8c78e81e6", "title": "Inception", "release_year":
2010, "rating": 9.2, "box_office": 836.8 }, { "id":
"6a3443c1c25310c8c78e81e8", "title": "Dune: Part Two", "release_year":
2024, "rating": 9.1, "box_office": 712.4 }, { "id":
"6a3443c1c25310c8c78e81e9", "title": "Interstellar", "release_year":
2014, "rating": 8.9, "box_office": 731 } ], "movie_count": 3, "genre":
"Sci-Fi" }, { "genre_id": null, "movies": [ { "id":
"6a34456fad642bb3eac7b165", "title": "Memento", "release_year": 2000,
"rating": 9, "box_office": 40 } ], "movie_count": 1, "genre":
"Uncategorized" } ]
```



3. GET /api/reports/movies-by-director (Movies Grouped by Director)

- **Response JSON**:

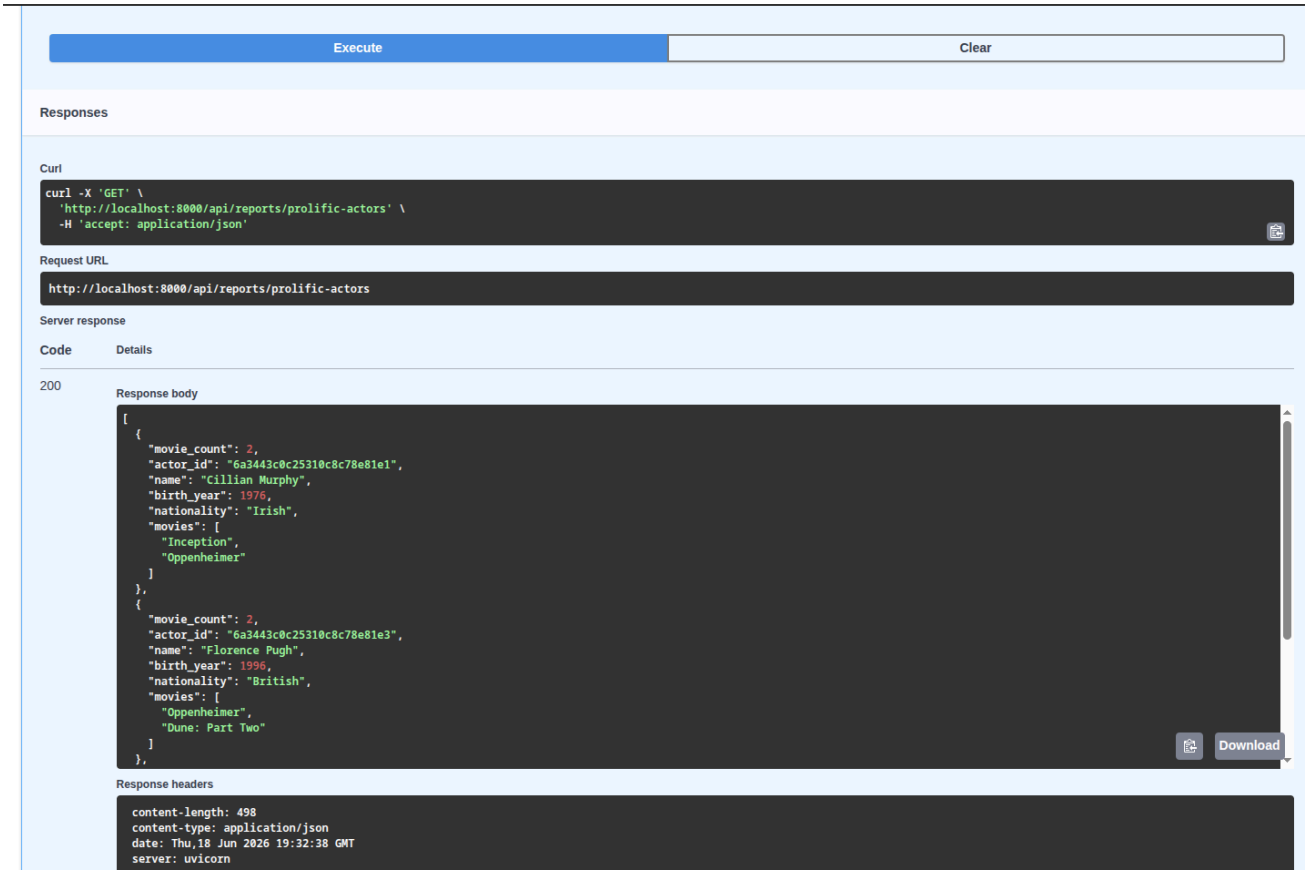

```
[ { "id": "6a3443c0c25310c8c78e81dd", "name": "Christopher Nolan",
"birth_year": 1970, "nationality": "British-American", "director_id":
"6a3443c0c25310c8c78e81dd", "movies": [ { "id":
"6a3443c0c25310c8c78e81e6", "title": "Inception", "release_year": 2010,
"rating": 9.2, "box_office": 836.8 }, { "id": "6a3443c1c25310c8c78e81e7",
"title": "Oppenheimer", "release_year": 2023, "rating": 9.1,
"box_office": 957 }, { "id": "6a3443c1c25310c8c78e81e9", "title":
"Interstellar", "release_year": 2014, "rating": 8.9, "box_office": 731 }
], "movie_count": 3 }, { "id": "6a3443c0c25310c8c78e81de", "name": "Denis
Villeneuve", "birth_year": 1967, "nationality": "Canadian",
"director_id": "6a3443c0c25310c8c78e81de", "movies": [ { "id":
"6a3443c1c25310c8c78e81e8", "title": "Dune: Part Two", "release_year":
2024, "rating": 9.1, "box_office": 712.4 } ], "movie_count": 1 }, { "id":
"6a3444a2ad642bb3eac7b162", "name": "Martin Scorsese", "birth_year":
1942, "nationality": "American", "director_id":
"6a3444a2ad642bb3eac7b162", "movies": [], "movie_count": 0 }, { "id":
"6a3443c0c25310c8c78e81df", "name": "Quentin Tarantino", "birth_year":
1963, "nationality": "American", "director_id":
"6a3443c0c25310c8c78e81df", "movies": [], "movie_count": 0 } ]
```



4. GET /api/reports/prolific-actors (Actors in Multiple Movies)

- **Response JSON**:

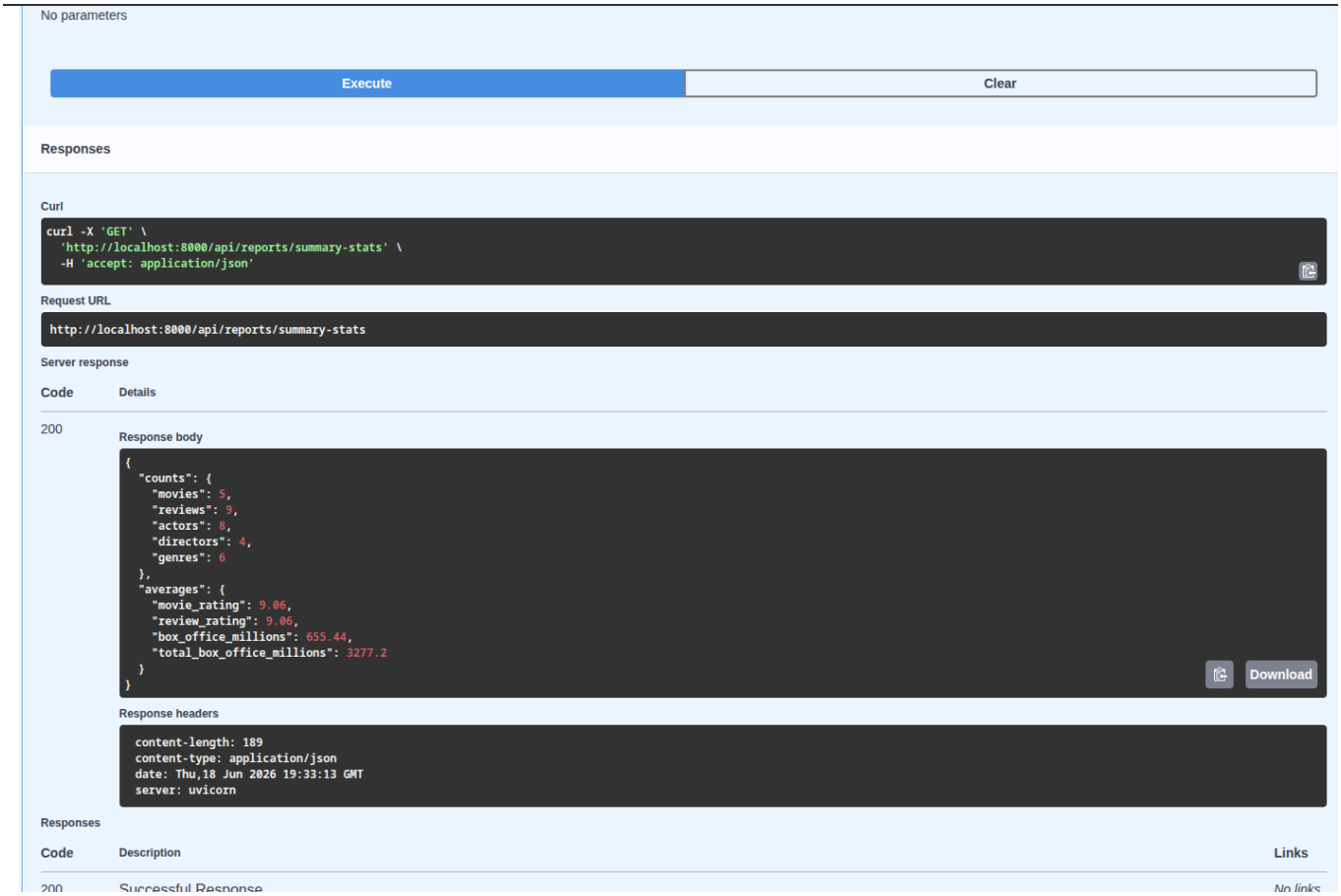

```
[ { "movie_count": 2, "actor_id": "6a3443c0c25310c8c78e81e1", "name": "Cillian Murphy", "birth_year": 1976, "nationality": "Irish", "movies": [ "Inception", "Oppenheimer" ] }, { "movie_count": 2, "actor_id": "6a3443c0c25310c8c78e81e3", "name": "Florence Pugh", "birth_year": 1996, "nationality": "British", "movies": [ "Oppenheimer", "Dune: Part Two" ] }, { "movie_count": 2, "actor_id": "6a3443c0c25310c8c78e81e2", "name": "Timothée Chalamet", "birth_year": 1995, "nationality": "American-French", "movies": [ "Dune: Part Two", "Interstellar" ] } ]
```



5. GET /api/reports/summary-stats (Overall Summary Stats)

- **Response JSON**:


```
{ "counts": { "movies": 5, "reviews": 9, "actors": 8, "directors": 4,
"genres": 6 }, "averages": { "movie_rating": 9.06, "review_rating": 9.06,
"box_office_millions": 655.44, "total_box_office_millions": 3277.2 } }
```



6. Summary and Conclusion

Schema Design & Modeling

In this system, we used a **Reference-Based Relational Modeling** strategy for core entities, combined with a **Nested Array Modeling** approach for Many-to-Many associations.

- **Directors, Genres, and Reviews** reference movies through standard ObjectIds (`director_id` , `genre_id` , `movie_id`). This keeps document sizes small and avoids data duplication.
- **Many-to-Many Actors & Movies** is modeled using an `actor_ids` list directly inside the Movie document. This is highly performant in MongoDB, avoiding complex junction tables.

Aggregation Pipeline Strengths

The project demonstrates the strength of **MongoDB Aggregations** to run analytical queries on the server. Groupings, joins, calculations, and unwinding are processed on the database server, sending only the final result sets back to the API. This reduces network payload and improves client-side speed.

Conclusion

This project successfully shows how to integrate an asynchronous Python stack (**FastAPI** + **Motor**) with **MongoDB**. By exposing OpenAPI specifications via **Swagger UI**, the project satisfies certification requirements with an interactive testing dashboard.